

多核 Ring 总线 ESL 模型

架构与详细设计

H/V 多 Ring 拓扑、核心架构与接口
活性保障、包级时序与存储侧聚合

2026 年 9 月 6 日

目录

摘要.....	4
设计特性一览.....	4
1 设计背景与目标.....	5
1.1 项目背景.....	5
1.2 设计目标.....	5
1.3 模型边界与术语.....	5
2 总体架构.....	7
2.1 分层结构.....	7
2.2 H/V Ring 组织.....	7
2.3 Ring 域和路由.....	8
3 协议与事务路径.....	10
3.1 子网与载荷.....	10
3.2 读写事务.....	10
4 互连核心模块.....	12
4.1 模块职责.....	12
4.2 Cross Station 调度.....	12
4.3 Conn 与 Slot Pool.....	13
4.4 RBRG 跨环转发.....	13
5 存储侧聚合.....	15
5.1 HA、L2 Buffer 与 TmMem 边界.....	15
5.2 两级聚合与返回.....	15
6 接口设计.....	17
6.1 统一接口语义.....	17
6.2 互连与拓扑接口.....	17
6.3 RBRG 与存储接口.....	18
7 仲裁活性与时序.....	19
7.1 I-tag、E-tag 与偏转.....	19
7.2 包级时序.....	20
8 地址映射与关键参数.....	21
8.1 地址到目标端的映射.....	21
8.2 关键参数.....	21
9 性能观测与验证.....	23
9.1 PMU 诊断.....	23
9.2 验证要点.....	23
9.3 L2 命中率对带宽的影响.....	24
9.4 并发请求数量与资源压力.....	25

9.5 请求大小与数据聚合收益.....	26
10 设计总结.....	29

摘要

本文面向海思 NPU 部门的多核总线性能探索，构建 ESL 模型，用于分析计算核规模、拓扑组织、链路带宽、队列容量和存储访问模式对系统性能的影响。模型采用多条垂直环汇聚至共享水平环的分层结构，将局部通信限制在较小的垂直环（V-Ring）内，并通过环间桥接模块（RBRG，沿用源码模块名称）接入水平环（H-Ring）和共享存储端。

模型以包级槽位（packet-slot）作为调度与计时抽象。请求、完成响应和数据分别使用 REQ、RSP、DAT 三类子网；存储侧由归属代理（Home Agent，HA）合并同一缓存行的读取请求，再由 L2 Buffer 按照目标 V-Ring 组织单播、多播或分散返回。本文重点说明系统架构、模块接口、事务路径、活性机制、时序模型、地址映射和关键参数。该模型用于事务流程验证和性能趋势分析，不替代寄存器传输级（Register Transfer Level，RTL）实现或完整的微片（flit）级片上网络模型。

设计特性一览

表 0.1 设计特性一览

维度	设计方案	作用
拓扑	一条共享 H-Ring 与多条自动分组 V-Ring	限制单环节点数和局部竞争范围
流量	REQ、RSP、DAT 独立子网，CW/CCW 双向传输	隔离控制包、完成响应和数据包的资源占用
调度	先进先出队首、Transit/Local 轮询仲裁	保持顺序并在环上转发与本地注入之间提供公平性
跨环	每条 V-Ring 配置一个 RBRG	在 H/V 域之间重建路由并重新支付序列化开销
活性	槽位池、I-tag、E-tag 与偏转协同	避免共享弹出队列和闭环反压形成永久等待
存储	HA 读合并与 L2 多接收端扇出	减少重复后端读取和重复 DAT 物理传输
时序	每个 Ring 段仅在首跳计入一次包序列化延迟	保留多跳观察延迟，每条 Conn 仍独立占用序列化周期

1 设计背景与目标

1.1 项目背景

随着 NPU 中 AI Core 数量增加，互连需要承载更多地址请求、写数据、读返回和完成响应。若将全部端点连接到同一环形互连（Ring），节点（station）数量和平均跳数将随系统规模增长，共享链路和队列的竞争也会加剧；局部流量热点还可能通过反压扩散至整个 Ring。若直接在 RTL 阶段反复调整拓扑、队列和链路带宽，修改范围和验证成本较高，不利于前期快速比较不同架构参数。

本项目据此构建可配置、可观测的 Ring ESL 模型，用于识别流量汇聚位置、首要饱和和资源以及参数变化引起的瓶颈迁移，并为拓扑选择、带宽配置和队列深度设计提供依据。H/V 分层拓扑将相邻计算核组织在多条 V-Ring 内，共享存储端部署在 H-Ring 上；跨环事务统一经过 RBRG，从而划分局部通信域与共享存储访问路径。

1.2 设计目标

模型需要完整描述从主设备（Master，请求发起端）到目标端（Target，存储服务端）的事务路径，并保留以下行为：

- 根据 Master 数量和单环容量自动生成多条 V-Ring，并接入共享 H-Ring；
- 使用独立 REQ、RSP、DAT 子网描述读写请求、完成响应和数据传输；
- 在交叉节点调度模块（Cross Station，CS）中实现双向最短路由、环上转发与本地注入仲裁；
- 通过链路连接模块（Conn）描述包序列化与逐跳传播，并通过 RBRG 的换环队列和串行化资源描述跨环开销；
- 在 HA 和 L2 Buffer 路径中描述读合并、写冲突、单播、多播和分段返回；
- 通过 PMU 定位链路、队列、RBRG、HA、L2 和内存后端的瓶颈。

1.3 模型边界与术语

表 1.1 模型术语与边界

名称	本文含义
H-Ring	连接 RBRG、HA、内存端口和 L2 Buffer 的共享水平环
V-Ring	连接一组 Master 与一个 RBRG 垂直端的局部环
Ring 段（segment）	一个包在同一 Ring 域内从注入到弹出的连续传输区间
节点接口（Node Interface，NI）	保存本地注入队列和目的弹出队列的端点接口
槽位池（Slot Pool）	记录一个 Ring 域内可注入包槽位容量的资源计数器

名称	本文含义
I-tag、E-tag	分别保护本地注入槽位和目标弹出容量的活性标记
承载包 (carrier)	在 H/V Ring 上传输一个或多个逻辑读响应的物理数据包
等待项 (waiter)	合并到同一 HA 事务、等待共享完成数据的逻辑读请求
内存可接收额度 (credit)	内存后端允许同时在途的请求数量，与 Ring 槽位相互独立

2 总体架构

2.1 分层结构

如表 2.1 所示，模型按照事务处理、存储服务、互连控制、物理时序和性能观测五类职责进行分层。各层通过明确的事务接口和状态转移条件连接，运行状态由对应模块统一维护，从而使拓扑、链路、队列和存储后端参数能够独立调整，同时保持端到端事务语义不变。

表 2.1 系统分层与模块职责

层次	主要模块	核心职责
事务接入层	Master NIU、内存端口	在外部事务接口与 Ring 载荷之间转换
存储服务层	HA、L2 Buffer、TmMem	读合并、写冲突保护、后端访问、完成数据暂存与扇出
互连控制层	Fabric、Topology、Cross Station、RBRG	构造网络、地址定位、仲裁、路由和跨环转发
物理近似层	Node Interface、Conn、Slot Pool	队列容量、链路序列化、传播延迟和反压
可观测层	PMU 与报告工具	汇总队列、链路、阻塞、延迟、吞吐和聚合统计

2.2 H/V Ring 组织

设系统中的主设备（Master）总数为 M ，单条垂直环（V-Ring）最多容纳 C 个 Master，存储目标端数量为 T ，则系统所需的 V-Ring 数量为：

$$V = \text{ceil}(M/C)$$

其中，ceil 表示向上取整。Master 按照每条 V-Ring 最多容纳 C 个节点的约束依次分组，最后一条 V-Ring 中的 Master 数量可以少于 C 。每条 V-Ring 由本环 Master 节点和一个 RBRG 垂直端组成，因此其节点数为：

$$\text{每条 V-Ring 节点数} = \text{本环 Master 数量} + 1$$

每条 V-Ring 的节点 0 固定为 RBRG 垂直端，Master 从节点 1 开始顺序排列。H-Ring 包含各 V-Ring 对应的 RBRG 水平端，以及每个目标端对应的 HA/内存端口和 L2 Buffer，因此其节点数为：

$$\text{H-Ring 节点数} = V + 2 \times T$$

以 8 个 Master、4 个目标端、每条 V-Ring 最多容纳 4 个 Master 为例，系统共生成 2 条 V-Ring。每条 V-Ring 包含 4 个 Master 节点和 1 个 RBRG 垂直端，共 5 个节点；H-Ring 则包含 2 个 RBRG 水平端、4 个 HA/内存端口和 4 个 L2 Buffer，共 10 个节点。

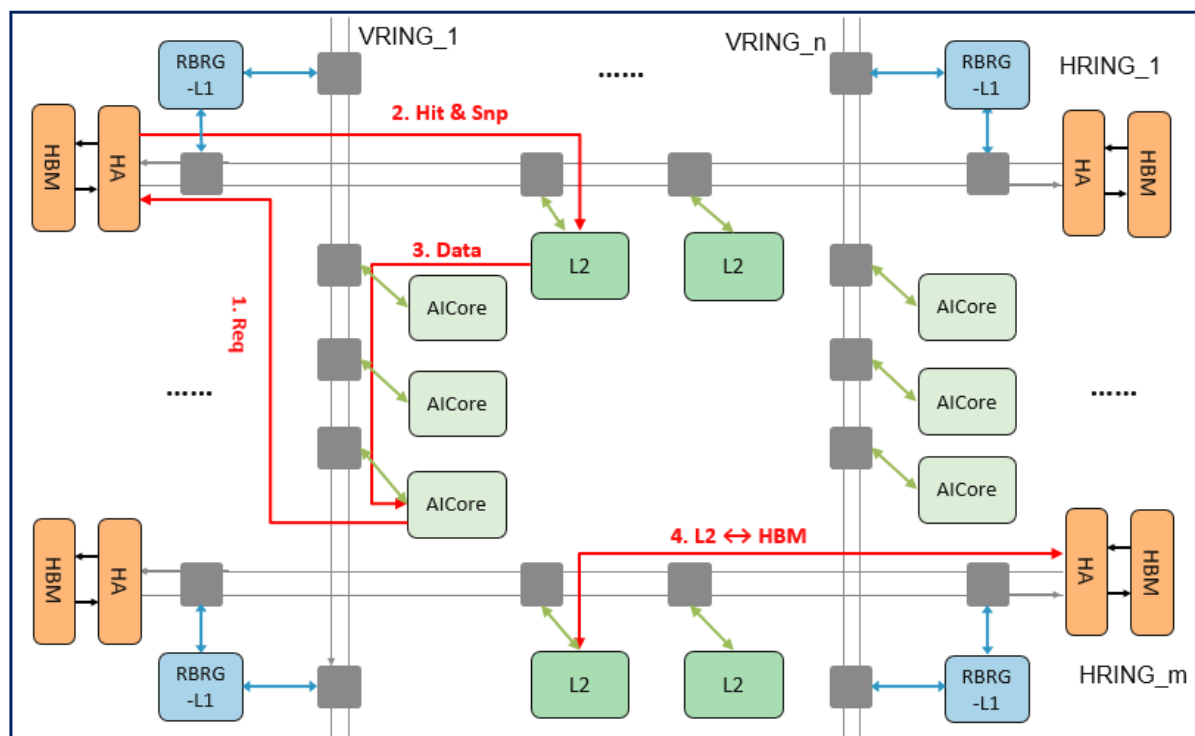


图 2.1 H/V 多 Ring 总体拓扑

如图 2.1 所示，计算核心按照容量约束划分至多条 V-Ring，HA、L2 Buffer 及存储后端连接在共享 H-Ring 上。请求包由本地 V-Ring 进入 RBRG，经换环后到达目标 HA；完成响应和读取数据则以 Master 为新目标，在 H-Ring 和目标 V-Ring 中分别重新计算路由后返回。该结构将局部竞争限制在各 V-Ring 内部，同时为所有计算核心提供统一的共享存储访问路径。

2.3 Ring 域和路由

H-Ring 与每条 V-Ring 均构成独立的双向包槽位域，各自包含 Cross Station、顺时针（Clockwise, CW）与逆时针（Counter-Clockwise, CCW）有向 Conn、Slot Pool 以及 REQ、RSP、DAT 三类子网。三类子网可以并行推进，同一子网的 CW 和 CCW 方向则使用相互独立的传输资源。

拓扑模块（Topology）保存 Master、HA、L2 和 RBRG 的位置，并在同一 Ring 域内选择最短传输方向；当 CW 和 CCW 距离相同时，固定选择 CW，以保证仿真结果可复现。跨域数据包仅在 RBRG 换环时结束当前 Ring 段，并根据目标 Ring 中的源位置和目的位置重新确定方向及序列化状态。

表 2.2 关键架构选择

设计选择	原因	约束
多条 V-Ring 汇聚到一条 H-Ring	缩小局部竞争域，并保留统一存储路径	跨域访问必须经过 RBRG
固定双向最短路由	状态少、路径确定、便于性能对比	不根据实时拥塞动态改道，拥塞由反压、偏转和活性机制处理
REQ/RSP/DAT 独立子网	防止地址、完成响应和大数据包直接互相阻塞	三类流量分别配置队列和链路统计
每个 Ring 域独立 Slot Pool	明确容量归属和复位边界	跨环后需要在新域重新申请槽位
包槽位时序近似	保留包长、链路宽度、跳数和反压的主要影响	不分析微片交错、虚通道和尾微片行为

3 协议与事务路径

3.1 子网与载荷

Ring 载荷由端到端事务字段和 Ring 段局部字段组成。主设备标识 `mst_id`、事务标识 `gid`、地址 `addr`、数据长度 `size` 和命令类型在事务生命周期内保持不变；传输方向、I-tag、E-tag、偏转状态以及本段序列化支付状态仅在当前 Ring 段内有效，RBRG 建立新段时需要重新初始化。

表 3.1 Ring 子网与载荷

子网	主要命令	典型方向	关键载荷
REQ	读请求 RD、写请求 WR	Master 到目标端	发起者、事务标识、地址、长度、目标端
RSP	完成响应 RSP	目标端到 Master	原事务身份和完成状态
DAT	写数据 WR_DAT、读数据 RD_RSP	双向	原事务身份、数据缓冲区和承载包信息

当前模型将控制类包按 16 B 计算 Ring 物理传输量，数据类包则按实际数据长度计算。写请求与写数据使用主设备标识和事务标识组成的联合键（`mst_id`, `gid`）进行配对；中间转发模块只能读取稳定的事务身份，不得在传输过程中重新编号。

3.2 读写事务

读请求依次经过 Master 网络接口单元（Network Interface Unit, NIU）、本地 V-Ring、RBRG、H-Ring 和 HA。首次到达某一缓存行的请求创建 HA 事务，后续同缓存行请求在事务表项和等待项仍有容量时合并。对于新建事务，模型按照配置的 L2 命中规则进行分类：命中时执行一次功能读取，未命中时仅向 TmMem 提交一次完整缓存行读取。完成数据进入 L2 Buffer 后，先按照目标 V-Ring 分组，再经 H-Ring 和 RBRG 返回各 Master。

写请求 WR 和写数据 WR_DAT 位于不同子网，可以在不同周期到达。内存端口分别检查两个来源队列的队首，并通过联合事务键（`mst_id`, `gid`）完成配对；只有后端成功接纳写事务后，两个队首才同时出队，最终生成一次完成响应。同一缓存行的写事务完成前，HA 持续保持写冲突状态，阻止冲突读取越过尚未完成的写操作。

表 3.2 事务提交点与反压行为

阶段	成功提交条件	阻塞时保持的状态
Master 注入	目标方向 Inject Queue 有容量且 Slot Pool 可分配	上游载荷和本地队首不变

阶段	成功提交条件	阻塞时保持的状态
Ring 单跳	Conn 串行化占用单元和流水可接收	Transit/Local 队首和仲裁指针不变
RBRG 换环	目标 Ring 的方向注入队列有容量，桥接串行化资源空闲	来源弹出队首和桥接载荷不变
HA 接纳	活跃读表和等待项有容量，且无写冲突	内存端口保留请求队首
后端提交	TmMem 带宽与 credit 允许	候选请求不提交，等待后端事件
L2 返回	响应缓冲和发射节拍允许	HA 等待项或 L2 承载包保持
最终完成	Master 消费 RD_RSP 或 RSP	目的弹出队列继续反压上游

各阶段只有在下游成功接纳载荷后才提交状态转移；发生反压时，来源队首、载荷内容和仲裁状态均保持不变。因此，阻塞不会造成数据包丢失、队首跳过、重复统计或仲裁指针提前推进。

4 互连核心模块

4.1 模块职责

表 4.1 互连模块职责

模块	主要状态	功能边界
Fabric	拓扑对象、模块实例和绑定关系	校验配置，构造 H/V Ring，管理复位、空闲和 PMU 快照
Topology	位置、地址到目标端映射、相邻关系	生成节点并给出同域最短路由，不维护运行时队列
Master NIU	外部事务适配状态	生成第一段路由并将返回包交给 Master 接口
Node Interface	方向注入队列和弹出队列	保存端点队列，不执行 Transit/Local 仲裁
Cross Station	输入寄存器、仲裁指针和活性标记	处理环上转发、本地注入、目的弹出和偏转
Conn	各子网串行化占用和固定延迟流水	接收包槽位并在到期后写入下游输入寄存器
Slot Pool	每个子网和方向的槽位占用	只记录容量，不保存载荷
RBRG	四类路径的方向队列和串行化状态	结束当前 Ring 段并在目标 Ring 建立新段

4.2 Cross Station 调度

每个 Cross Station 按照“子网×输入方向”维护一个深度为 1 的输入寄存器（Reg_In），并按照“子网×输出方向”维护一组 Transit/Local 轮询仲裁。Transit 表示来自对应上游 Reg_In 并沿既定方向继续传输的环上数据包，Local 表示同一输出方向 Inject Queue 的队首数据包。到达目的节点的数据包优先尝试进入共享 Eject Queue；若弹出队列暂时无可用容量，则按照 E-tag 规则继续绕行。

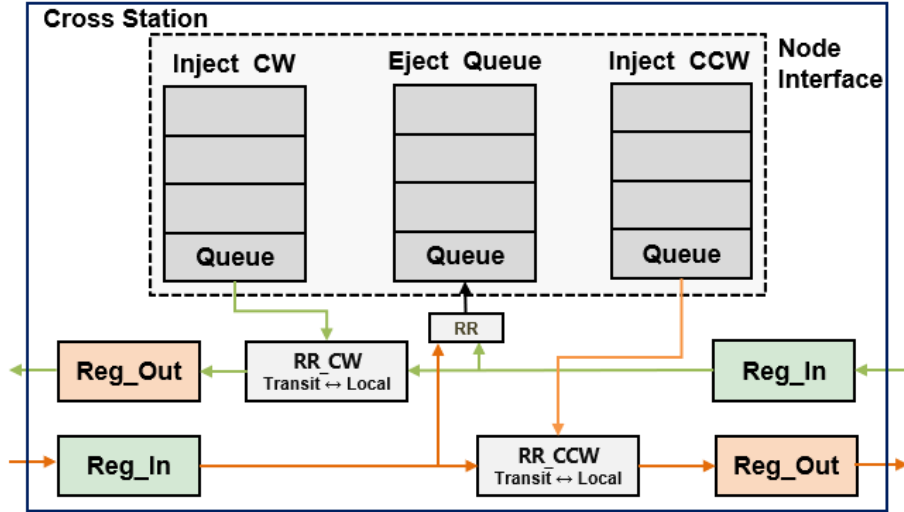


图 4.1 Cross Station 端口、队列与双向仲裁

如图 4.1 所示，CW 和 CCW 方向分别配置独立的 Transit/Local 仲裁器，因此可以在同一周期处理两个方向的候选数据包。共享 Eject Queue 仅接收已经到达本站的目标包，不构成仲裁器的第三个输入。当本地只有一个注入来源时，轮询仍用于协调环上转发与本地注入两类候选。来源候选仅在 Conn 成功接纳后出队，轮询状态也在此时推进。

4.3 Conn 与 Slot Pool

Conn 表示相邻 Cross Station 之间的一条有向链路。REQ、RSP、DAT 分别拥有独立的串行化占用单元和固定延迟流水。若串行化资源忙、流水已满或下游 Reg_In 被占用，包保持在原有位置，等待相应空间事件后重试。

Slot Pool 仅记录一个 Ring 域内各“子网×方向”的逻辑槽位占用，不保存载荷。设 Ring 节点数为 N 、链路传播延迟为 L ，则链路流水深度 $D = \max(1, L + 1)$ ，每个子网和方向的槽位容量为 $N \times (D + 1)$ 。模型始终保留一个空槽，因此可注入的数据包数量上限为 $N \times (D + 1) - 1$ ，使闭环反压能够继续向上游传播。数据包在本地成功注入前申请槽位，在最终弹出或完成槽位替换时释放。Slot Pool 用于判断数据包能否进入 Ring，Conn 则决定数据包何时到达下一节点。

4.4 RBRG 跨环转发

RBRG 支持四类合法协议路径：V-Ring 到 H-Ring 的 REQ、V-Ring 到 H-Ring 的 DAT、H-Ring 到 V-Ring 的 RSP，以及 H-Ring 到 V-Ring 的 DAT。每类路径按照目标 CW 或 CCW 方向配置独立队列和串行化资源。当同一路径同时接收到来自源 Ring 两个方向弹出队列的候选数据包时，RBRG 仅在两个队首之间进行轮询选择，不越过队首扫描队列内部。

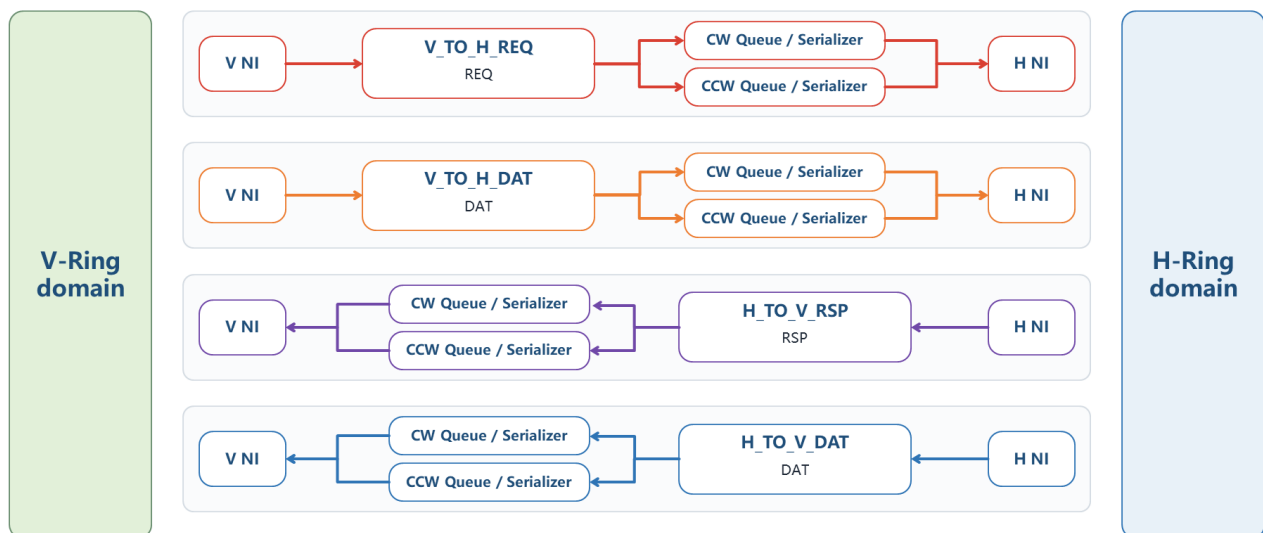


图 4.2 RBRG 四类协议路径与方向化资源

如图 4.2 所示，RBRG 的垂直侧和水平侧分别连接两个独立 Ring 域。只有当目标方向 Inject Queue 具有可用容量、对应桥接串行化资源空闲且目标 Ring 路由能够建立时，数据包才从来源 Eject Queue 弹出。换环后，端到端事务身份保持不变；当前 Ring 段的方向和序列化状态重新计算，I-tag、E-tag 及偏转等局部控制状态则被清除，避免将前一 Ring 段的控制状态带入新环。

5 存储侧聚合

5.1 HA、L2 Buffer 与 TmMem 边界

HA 按缓存行基地址维护活跃读事务。首笔读取创建表项，后续同缓存行读取成为等待项，共享一次功能读取或后端时序读取。表项从等待后端、后端在途到逐个返回期间持续占用，只有全部等待项完成并且相关扇出组回收后才释放。写事务则通过缓存行冲突状态保护未完成写，避免读到未更新的数据。

L2 Buffer 保存完成数据和扇出状态，并施加固定响应延迟、缓冲深度和最小发射间隔。TmMem 保存功能数据并提供后端延迟、带宽和 credit。三者的容量相互独立：HA 表项限制可跟踪缓存行数，L2 缓冲限制待返回完成数据，TmMem credit 限制后端在途请求，均不能用 Ring Slot Pool 替代。

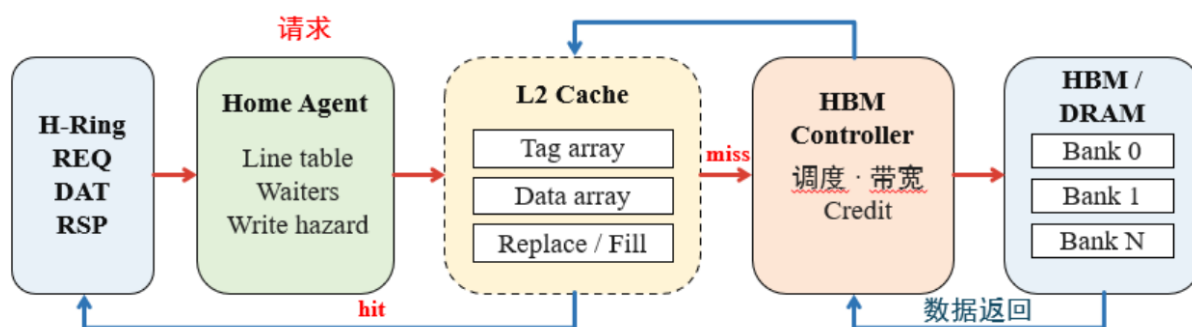


图 5.1 存储层概念路径与当前模型边界

如图 5.1 所示，目标存储路径在体系结构上包含 HA、L2 层和内存后端。图中的 L2 Cache 表示目标体系结构中的缓存层次；当前 ESL 实现不展开标签阵列、数据阵列和替换过程，仅通过 L2 Buffer 描述完成数据暂存、扇出组织、固定响应延迟和发送节拍。HA 负责事务合并，L2 Buffer 负责返回组织，TmMem 负责后端服务，三者分别维护各自的运行状态。

5.2 两级聚合与返回

表 5.1 HA 与 L2 两级聚合

阶段	输入	处理	输出
HA 合并	同一缓存行的多个逻辑读请求	首笔建立事务，后续追加等待项	一次功能读取或一次完整缓存行后端读取
L2 分组	共享完成数据和多个等待项	按目标 V-Ring 分为若干组	每个目标 V-Ring 一个 H-Ring 的承载包
V-Ring 展开	到达 RBRG 的组内接收端	同范围使用多播，不同偏移使用分散模式	各 Master 收到普通 RD_RSP

如表 5.1 所示，返回路径首先在 HA 完成事务聚合，再在 L2 Buffer 完成传输聚合，随后在目标 V-Ring 内展开。HA 以缓存行基地址为事务索引，将命中同一缓存行的读取请求合并到同一活跃事务中。首个请求创建事务并触发一次功能读取或后端读取，后续请求作为等待项追加，并保留各自的请求偏移、数据长度和返回目标。完成数据返回后，HA 将共享数据及等待项依次提交至 L2 Buffer，由 L2 Buffer 完成目标 V-Ring 分组和物理承载包生成。

L2 Buffer 根据请求端所属的目标 V-Ring 对等待请求进行分组，并为每个目标 V-Ring 生成一个物理承载包（carrier）。同一 V-Ring 内请求数据范围相同时采用多播方式共享载荷；分段或偏移不同时，则通过分散返回方式恢复各请求端所需的数据内容。承载包到达目标 RBRG 后在 V-Ring 内继续传输，各请求端依次接收对应响应，最后一个请求端完成接收后释放该承载包。

该机制分别在事务层和传输层消除重复操作：HA 合并降低后端读取次数，L2 分组减少跨 H-Ring 传输的物理承载包数量。其性能收益主要取决于请求地址的集中程度、同一事务的等待请求数、目标 V-Ring 数量以及返回数据量；当多个逻辑请求能够共享同一物理承载包时，可有效降低链路占用并缓解返回路径拥塞。

6 接口设计

6.1 统一接口语义

表 6.1 统一接口语义

接口类型	语义要求	失败处理
has/can/front/peek	只查询容量、队首或候选，不修改状态	返回失败后来源保持不变
push/accept/send	下游明确接收后转移载荷所有权	队列满或资源忙时返回失败，不出队
commit/pop	只提交已经被同一下游成功接受的候选	不允许跨候选、重复提交或提前推进仲裁
空间事件	队列由满变为可用时通知上游重新调度	不通过永久轮询掩盖遗漏的唤醒
复位接口	清空队列、串行化状态和事件代次	复位前事件不得推进新一轮状态

统一接口契约适用于 Master NIU、Node Interface、Cross Station、Conn、RBRG、HA 和 L2 Buffer。接口返回成功表示对应状态转移已经提交，而非仅表示请求已经发出；PMU 也只能在状态转移成功后更新相关计数。

6.2 互连与拓扑接口

表 6.2 互连与拓扑接口契约

边界或接口	主要调用	成功条件与状态变化	反压行为
外部 Master 到 Fabric	attach_master()及请求输入	索引合法并完成 Master NIU 绑定	未绑定或索引越界在构造阶段失败
目标端到 Fabric	attach_target()	目标端、HA、L2 和 TmMem 接口完成绑定	配置不完整时不进入运行阶段
Fabric 生命周期	build()、reset()、idle()	完成组网；复位全部状态；所有模块排空后返回空闲	idle()不得只检查 Ring 队列
Topology 定位	decode_target()、各类 *_location()	地址匹配目标范围和交织规则	非法索引或跨域位置明确失败
Topology 路由	route_direction()、neighbor_station()	源目的处于同一 Ring 域	跨域包交给 RBRG 建立新段

边界或接口	主要调用	成功条件与状态变化	反压行为
Node Interface 注入	has_inject_capacity()、push_inject()	目标方向队列有容量，载荷进入队列	返回失败，上游保留载荷
Node Interface 弹出	has_eject_capacity()、push_eject()	共享或方向弹出队列可接收	目标包继续沿环偏转
Cross Station 到 Conn	accept_slot()	串行化资源和流水可接受	来源队首与轮询状态保持
Conn 到 Cross Station	下游 Reg_In 写入	目标深度 1 寄存器为空	成熟包留在 Conn 流水中重试

6.3 RBRG 与存储接口

表 6.3 RBRG 与存储侧接口契约

边界或接口	主要数据	成功条件	状态所有者
RBRG 来源 Eject 到目标 Inject	REQ、RSP 或 DAT 载荷	目标方向队列有容量且桥接串行化资源空闲	RBRG 持有换环中的载荷
内存端口到 HA	读请求、写冲突通知	HA 返回已接纳、已合并或旁路	HA 维护活跃读表和写冲突
HA 到 TmMem	完整缓存行后端请求	TmMem 带宽与 credit 可用，且后端成功接纳请求	TmMem 维护后端在途额度
TmMem 到 HA	完成数据	对应活跃读事务仍有效	HA 维护等待项与共享完成数据引用
HA 到 L2 Buffer	响应候选和目标列表	L2 响应缓冲有容量	接受前由 HA 保持候选，接受后由 L2 持有
L2 Buffer 到 Node Interface	H-Ring DAT 承载包	发射节拍允许且 Inject Queue 可接收	L2 保留当前接收端分组和待发送数据包
Master NIU 到外部接口	RSP 或 RD_RSP	上层接口消费完成	未消费时 Eject Queue 保持并反压

外部接口仅暴露普通读写事务和完成响应；I-tag、E-tag、偏转、Ring 段方向和扇出状态均属于互连内部字段。跨模块共享载荷指针不代表共享修改权限，每个字段只能由当前协议阶段的责任模块更新。

7 仲裁活性与时序

7.1 I-tag、E-tag 与偏转

当本地注入队列存在待发送数据，而 Slot Pool 无法分配新的环形槽位时，Cross Station 会在经过本站且尚未被其他节点标记的环上槽位中设置 I-tag，用于记录本站的注入需求。该标记随槽位继续沿环传输；当槽位再次返回标记节点且本地队首仍在等待时，Cross Station 以本地数据包替换该槽位，并在注入成功后清除标记。当目标数据包到达本站而 Eject Queue 已满时，数据包保留原有 Ring 槽位并继续绕行，同时通过 E-tag 在对应弹出队列中预留一个表项；其他数据包不得占用该预留容量，携带 E-tag 的数据包再次到站后可优先弹出。两类标记分别保护注入槽位和弹出容量，从而避免闭环高占用条件下的长期等待。

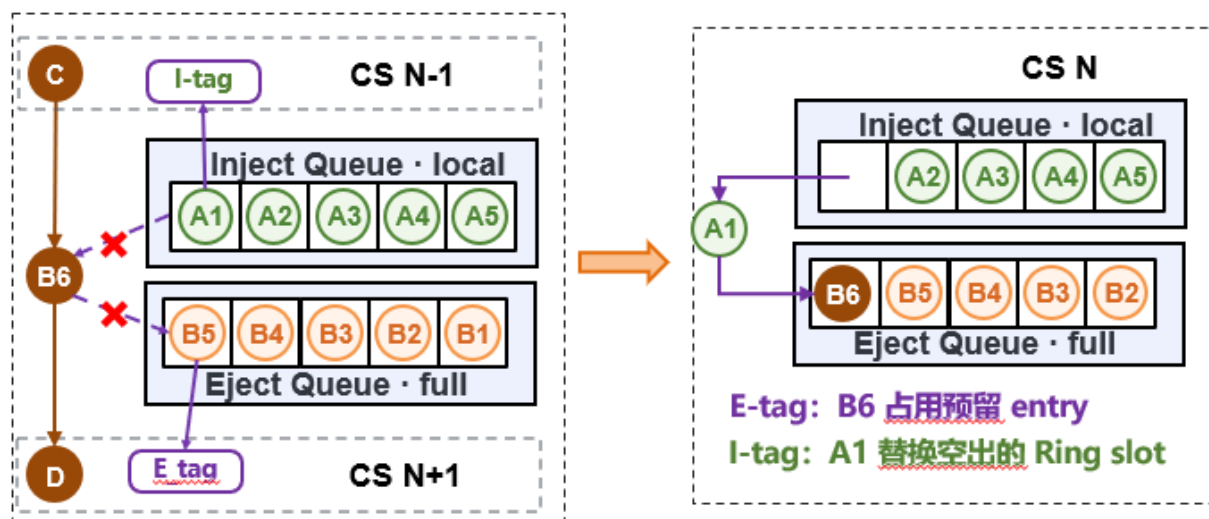


图 7.1 I-tag 与 E-tag 阻塞和恢复过程

如图 7.1 所示，左侧目标包 B6 到达 CS N 时，弹出队列已被 B5~B1 占满，因此 B6 无法立即弹出，只能携带 E-tag 继续沿环转发。与此同时，本地注入队列中的 A1 因 Slot Pool 当前无法分配新槽位，由 I-tag 保留后续注入机会。右侧弹出队列释放表项后，B6 再次到达 CS N，并使用 E-tag 预留的表项进入弹出队列；B6 释放出的 Ring 槽位随后由 A1 替换占用，从而完成本地注入。

I-tag 与 E-tag 分别解决注入侧和弹出侧的长期等待问题：I-tag 保证本地队首最终能够获得 Ring 槽位，E-tag 保证目的数据包最终能够进入弹出队列。二者保护的资源和触发条件不同，共同维持 Cross Station 在持续拥塞条件下的转发活性。

表 7.1 活性机制与释放条件

机制	解决的问题	生效位置	释放条件
Transit/Local 轮询	环上流量与本地流量长期偏置	每个子网和输出方向的 Cross Station	某一候选实际发送成功后推进

机制	解决的问题	生效位置	释放条件
I-tag	本地队首长期无法获得注入槽位	当前节点和当前 Ring 段	受保护槽位完成替换或被取消
E-tag	目的 Eject Queue 已满导致包无法弹出	目的节点和绕行包	预留表项被目标包消费
偏转	目的端暂时无容量	包的当前方向	后续到站成功弹出
Slot Pool 保留空槽	闭环完全占满后反压无法传播	每个 Ring 域、子网和方向	包最终弹出或完成槽位替换

7.2 包级时序

为在不展开 flit 级传输状态的前提下描述链路带宽和多跳传播开销，本模型采用包级直通近似，并以一段连续的 Ring 链路作为序列化计时边界。设数据包大小为 P 字节，链路宽度为 W B/cycle，每跳固定传播延迟为 L 周期，当前 Ring 段包含 H 跳，则数据包的序列化周期 S 和该段传输延迟 $T_{segment}$ 分别为：

$$S = \max[1, \text{ceil}(P/W)]$$

$$T_{segment} = (S - 1) + H \times L$$

其中， $\text{ceil}(P/W)$ 表示对 P/W 的计算结果向上取整，即数据包完整通过链路所需的周期数。数据包进入当前 Ring 段的第一条链路 Conn 时，需要支付一次序列化延迟 $S-1$ 和首跳传播延迟 L ；经过交叉站点继续向后续 Conn 转发时，每增加一跳仅在数据包到达时间上叠加传播延迟 L ，不再重复计算当前段的序列化延迟。

包级直通近似只缩短数据包在相邻节点之间的交接延迟，不减少实际链路资源占用。数据包经过的每条 Conn 仍需独立占用 S 个周期，在占用结束前不能接收下一个完整数据包。因此，该模型能够保留包大小、链路宽度、传播跳数及链路竞争对性能的主要影响。

当数据包通过 RBRG 从当前 Ring 进入另一条 Ring 时，原有连续传输段结束，并从换环位置开始建立新的传输段。数据包需要根据 RBRG 的数据通路宽度重新计算换环序列化开销；进入目标 Ring 后，再按照目标 Ring 的链路宽度和传播跳数计算新段延迟。

以 512 字节数据包、128B/cycle 链路宽度、两跳传输和 1 周期单跳传播延迟为例，序列化周期 $S=4$ ，该 Ring 段的传输延迟为：

$$T_{segment} = (4 - 1) + 2 \times 1 = 5 \text{ 周期}$$

该结果表示数据包到达第二跳输入寄存位置的链路传输时间，不包含目标端弹出、接口转换及上层模块处理所引入的额外延迟。

8 地址映射与关键参数

8.1 地址到目标端的映射

地址映射决定请求进入哪个 HA、L2 Buffer 和 TmMem。模型首先按照目标地址范围筛选候选目标端，再根据配置的交织方式选择切片。采用异或交织（XOR interleave）时，模型将缓存行号分别与哈希种子及移位后的高位行号进行异或运算，最后对目标端数量取模。

$$\begin{aligned}
 home_line &= (addr - addr_begin) / interleave_size \\
 hashed &= home_line \text{ XOR } seed \\
 &\quad \text{XOR } (home_line \gg shift) \\
 &\quad \text{XOR } (home_line \gg (2 \times shift)) \\
 slice &= hashed \% interleave_num
 \end{aligned}$$

当前设计以 512 B 缓存行为 HA 归属和目标端选择粒度，以 128 B 扇区（sector）作为 L2 承载包的最小对齐粒度。同一 512 B 缓存行内的四个 128 B 扇区必须映射到同一目标端；扇区编号只影响返回数据组织，不参与目标端选择。

8.2 关键参数

表 8.1 关键参数及影响

参数	结构体默认值或配置来源	直接作用	调整时的判断
num_masters	1	Master 数量和 V-Ring 分组	与单环容量共同决定 V-Ring 数量
max_aicore_per_vring	8	单条 V-Ring 最多 Master 数	过大会增加局部跳数，过小会增加 H-Ring 上的 RBRG 数量
ring_link_width_bytes	16 B/cycle	Ring 包序列化周期和链路服务率	加宽只改善 Ring 链路，不消除后端或 RBRG 瓶颈
ring_link_latency	1 cycle	每跳传播延迟和流水深度	主要影响单次延迟，不必然降低稳态吞吐
rbrg_width_bytes	128 B/cycle	换环序列化时间	与 Ring 链路宽度是两组独立资源
rbrg_queue_depth	4	跨环突发吸收能力	加深队列会延后反压，但不会提高稳定服务率
注入/弹出队列深度	配置指定（示例为 2）	端点短时突发和排队位置	应结合占用率、拒绝次数和队首等待分析

参数	结构体默认值或配置来源	直接作用	调整时的判断
HA 活跃表项	64	可同时跟踪的缓存行数	表项不足会阻塞新缓存行请求
每表项等待项	8	单缓存行可合并的逻辑请求数	增加合并能力，不增加 TmMem credit
L2 响应缓冲深度	16	完成数据暂存容量	过小会反压 HA，过大不改变 DAT 链路服务率
L2 响应延迟	20 cycles	完成数据可发射前的等待	用于建模目标层处理时延
缓存行/扇区	512 B/128 B	HA 合并粒度和 L2 承载包对齐	两者必须为 2 的幂，且缓存行可整除扇区
TmMem 带宽 /credit	由后端配置	后端服务率和在途深度	与 Ring Slot Pool 分别统计和调节

参数分析需要首先区分参数改变的是资源容量、稳定服务率还是单次传输延迟。队列深度和表项数量主要影响突发吸收能力及反压位置；Ring 链路宽度、RBRG 宽度和后端带宽直接影响服务率；传播延迟和 L2 响应延迟主要影响事务完成时间。只有提高首要饱和资源的服务率，系统稳态吞吐才会明显提升。

9 性能观测与验证

9.1 PMU 诊断

PMU 按照 Ring 域、子网和方向统计数据包数量、传输字节数、链路繁忙周期、队列占用、发送拒绝及阻塞原因，并分别记录 RBRG、HA、L2 和 TmMem 的运行状态。性能诊断从事务完成端沿数据路径反向展开：首先确认完成数量正确且各级数据路径已经排空，再定位最早出现持续阻塞的资源。

表 9.1 PMU 诊断顺序

检查顺序	主要指标	可判断的问题
Master 接入	外部队列、未完成事务数、Inject 占用与拒绝	请求源是否已被本地队列限制
V-Ring	方向利用率、Slot Pool、利用率最高的链路和偏转次数	局部环是否拥塞或方向不均衡
RBRG	来源 Eject 占用、串行化繁忙和目标 Inject 拒绝	换环带宽或目标环入口是否成为瓶颈
H-Ring	各子网物理字节、利用率最高的链路和节点弹出拒绝	共享环是否承担过多跨域流量
HA 与 L2	活跃表项、等待项、合并率、响应缓冲和承载包数	后端读取与返回聚合是否有效
TmMem	带宽利用、在途请求、credit 和服务阻塞	内存后端是否限制整体吞吐

9.2 验证要点

功能验证从拓扑和队列单元测试开始，并进一步覆盖 Cross Station 竞争、Conn 时序、RBRG 换环、HA/L2 聚合和端到端排空。关键通过条件包括：地址映射与路由结果符合预期；发送失败时，来源队首、轮询状态、活性标记和完成计数均不得推进；写请求与写数据按照事务身份正确配对；多播承载包在最后一个接收端完成后回收；仿真结束时，队列、串行化流水、Slot Pool、HA/L2 状态和待处理事件全部排空；逻辑响应数量、物理承载包数量和 DAT 传输字节数满足统计一致性约束。

性能对比应固定源码版本、配置、流量模式和统计区间，每组实验只改变一个主要参数，并同时观察吞吐、延迟和阻塞位置。队列加深主要改善短时突发吸收能力，不能据此直接判断稳定带宽得到提升。

9.3 L2 命中率对带宽的影响

实验采用四类地址访问模式：**NoMerge**（无合并访问）使各请求访问不同缓存行；**Private**（私有连续访问）使各请求端顺序读取独立地址区域；**Scatter**（同缓存行分散访问）使多个请求端读取同一缓存行内的不同数据段；**Shared**（共享访问）使多个请求端读取相同数据。这些模式用于区分缓存行内数据复用与跨核共享的影响。

以下带宽均以 B/cycle 计量。端到端有效带宽按各请求端实际收到的有效数据总量统计，HBM 物理带宽按存储后端实际读取的字节数统计，二者采用相同的传输时间区间。共享数据在多个请求端分别计入有效数据量，因此有效带宽不等同于单条物理链路的带宽。本组实验固定请求大小为 256 字节，将配置的 L2 命中率由 0% 逐步提高至 100%。

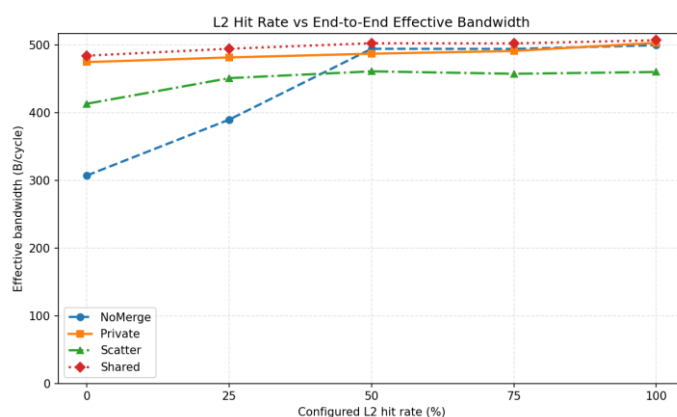


图 9.1 不同 L2 命中率下的端到端有效带宽

如图 9.1 所示，NoMerge 模式的有效带宽随命中率由 0% 提高至 50%，从 307.22 增至 494.49B/cycle，增幅约 61.0%；命中率继续提高至 100% 时，带宽仅增至 499.77B/cycle。Private 和 Shared 模式在低命中率下已获得较高带宽，后续增幅相对有限；Scatter 模式在 50% 附近达到约 461B/cycle，此后基本稳定。不同曲线进入平台的时机不同，说明访问模式与数据聚合共同影响系统对后端读取的依赖程度。

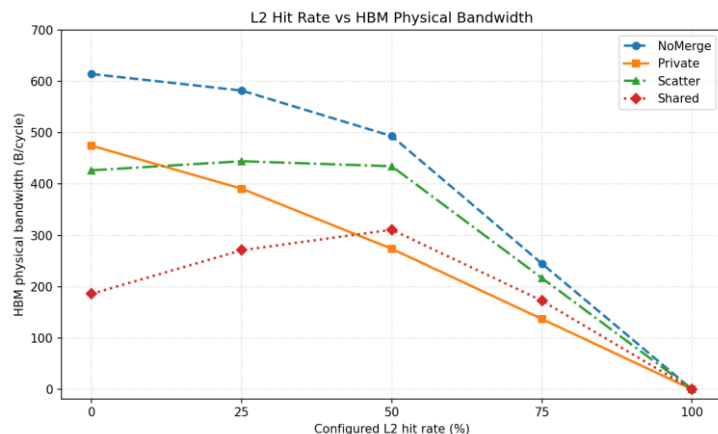


图 9.2 不同 L2 命中率下的 HBM 物理带宽

如图 9.2 所示，NoMerge 模式的 HBM 物理带宽由 614.44B/cycle 逐步降至 0。在 50% 至 100%命中率区间，有效带宽基本保持不变，而后端读取显著减少，表明继续提高命中率的主要收益已转为释放后端带宽；端到端性能更可能受数据返回路径的服务能力约束。仅凭这两张带宽图，尚不能将该约束进一步归因于某一条链路或某一个队列。

Shared 模式的 HBM 物理带宽在 0%至 50%区间由 186.15 升至 310.99B/cycle，并非单调下降。此时有效带宽仅由 484.33 升至 502.73B/cycle，完成速率的变化不足以单独解释这一现象。单位有效数据对应的后端读取量还受事务合并程度影响，不能仅按配置命中率线性推算。因此，共享场景应结合 HA 合并统计分析后端流量；各模式在 100%命中率下均不再产生 HBM 读取。

9.4 并发请求数量与资源压力

本组实验采用 256 字节的 NoMerge 读取，L2 命中率为 0%，将每个请求端允许同时 Outstanding 依次设为 16、32、64、128、256 和 512。该参数控制能够重叠执行的事务数量，用于观察并发增加能否覆盖链路而后端的等待时间。

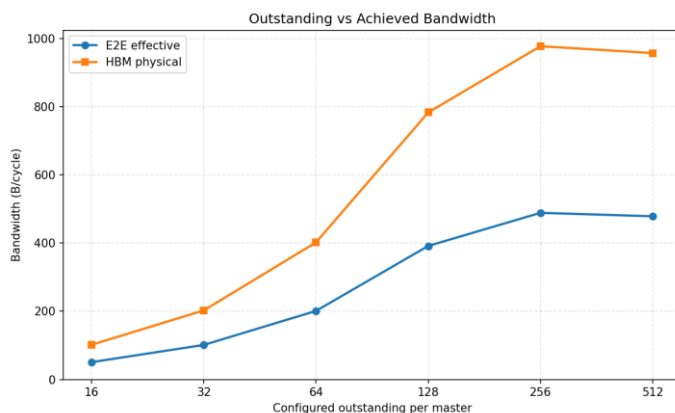


图 9.3 不同未完成请求上限下的有效带宽与后端带宽

如图 9.3 所示，并发上限由 16 增至 128 时，有效带宽从 50.79 增至 391.75B/cycle，总体接近随并发度增长；增至 256 后达到 488.88B/cycle，为本组已测点中的最高值。继续

增至 512 后，有效带宽回落至 478.80B/cycle，下降约 2.1%。这表明较低并发度下仍有未被利用的流水能力，而接近饱和后，增加请求数量不再带来吞吐收益。该结果支持以 256 作为本组配置的吞吐调优参考点，而非所有访问模式的通用设置。

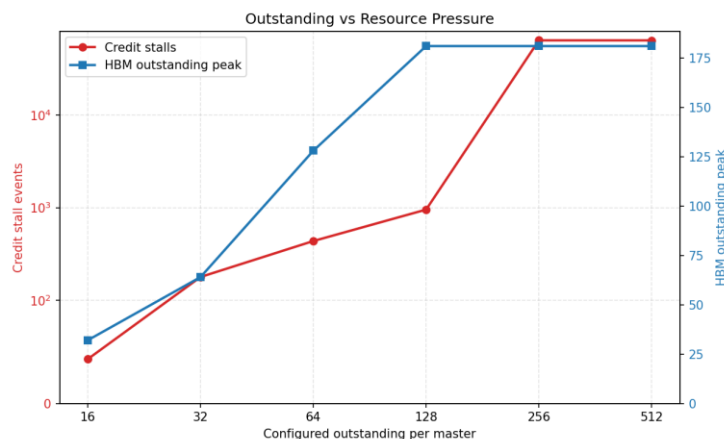


图 9.4 不同未完成请求上限下的后端资源压力

如图 9.4 所示，后端在途请求峰值在并发上限达到 128 后保持为 181；与此同时，后端 credit 不足的阻塞计数，从上限 128 时的 954 增至上限 256 时的 63922，在上限 512 时仍为 63882。该计数反映额度不足造成的阻塞压力，不等同于单笔请求的等待时间。结合带宽曲线可知，额外并发逐渐转化为资源竞争，而非更多有效传输。

需要注意，后端峰值在 128 至 256 区间未再增加，但有效带宽仍提高约 24.8%，说明瞬时在途峰值不能单独代表整条事务路径的利用程度。并发参数应结合有效带宽、后端阻塞及队列占用综合选择。此外，本组每次逻辑请求读取 256 字节，而后端按 512 字节完整缓存行读取，因此 HBM 物理带宽始终约为有效带宽的 2 倍。

9.5 请求大小与数据聚合收益

本组实验将 L2 命中率固定为 0%，每个请求端的未完成请求上限固定为 128，比较 128、256 和 512 字节请求的传输表现。在 128B/cycle 的链路宽度下，三者分别对应 1、2 和 4 拍数据。Scatter 仅测试 128 和 256 字节：当请求覆盖完整 512 字节缓存行时，不再具备同一行内不同数据段的分散访问条件。

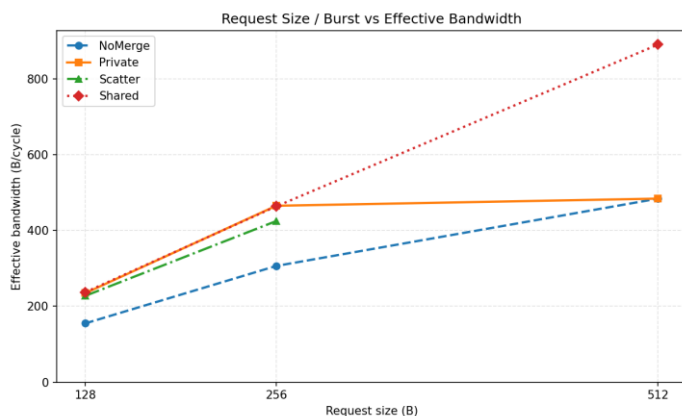


图 9.5 不同请求大小下的端到端有效带宽

如图 9.5 所示，NoMerge 模式的有效带宽随请求由 128 字节增至 512 字节，从 154.97 提高至 483.88B/cycle。Private 模式在 128 和 256 字节下分别达到 234.73 和 465.00B/cycle，均约为 NoMerge 的 1.52 倍；在 512 字节下，两者均为 483.88B/cycle。小请求下，连续访问能够复用同一缓存行的读取结果；请求增大至整行后，这部分优势随之缩小。与此同时，在有效数据总量相同的条件下，较大请求减少了请求发起与响应处理的次数。

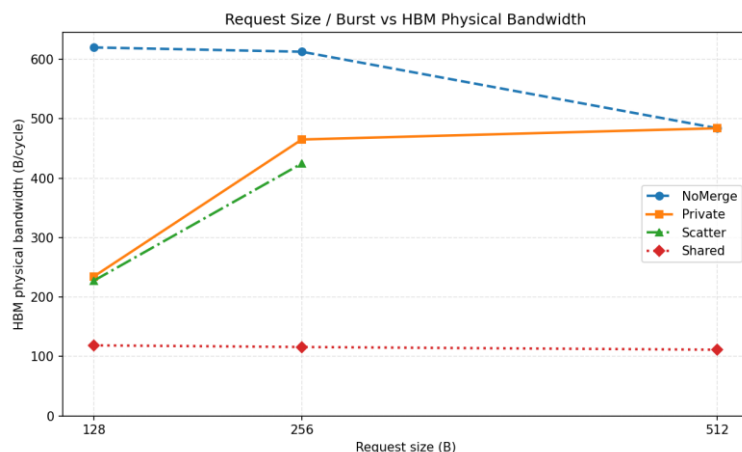


图 9.6 不同请求大小下的 HBM 物理带宽

如图 9.6 所示，NoMerge 模式的 HBM 物理带宽与有效带宽之比，在 128、256 和 512 字节请求下分别为 4、2 和 1。这与后端按 512 字节整行读取相对应：请求越小，未被本次请求使用的数据占比越高。因此，增大请求既能分摊事务处理开销，也能降低整行读取带来的额外数据搬运。

Shared 模式在 512 字节请求下达到 890.04B/cycle 的有效带宽，约为同尺寸 NoMerge 的 1.84 倍，而 HBM 物理带宽仅为 111.25B/cycle，两者之比为 8。对应统计表中的 L2 发出数据带宽为 222.51B/cycle，有效带宽与其之比为 4。前一比例体现多个请求端共用后端读取结果，后一比例体现返回数据通过多播服务多个接收端。该收益来自 HA 读合并与 L2 返回聚合，不代表单条物理链路的传输能力提高。

综合三组实验，性能调优应同时考虑命中率、并发度和访问粒度。带宽进入平台后，提高命中率主要减少后端压力；并发度应选择在吞吐收益开始收敛的位置；对于连续或共享访问，应优先利用缓存行内的数据复用及跨核返回聚合，减少重复读取与传输。

10 设计总结

本模型通过 H/V 分层拓扑、独立 REQ/RSP/DAT 子网、双向最短路由、RBRG 跨环转发以及 HA/L2 两级聚合，建立了面向多核 NPU 互连的包级事件驱动性能模型。模型在控制复杂度的同时保留了链路宽度、传播跳数、队列容量、反压和后端服务率等关键因素，可用于识别端到端事务路径中的首要饱和资源，并为拓扑选择、链路配置和存储侧参数设计提供分析依据。